

函数漏洞检测智能体工具

课程项目开题报告

组长：李柏谚　组员：杨聃、王义卓、孙伊凡、徐培栋

2026 秋季学期·高级软件工程



中国科学院大学

一、项目意义：我们为什么要做这件事

漏洞太多、老工具不好用，而大模型还缺一点「安全常识」

- **漏洞多到查不过来**：2026 年前两个季度新增的公开漏洞就超过 3.5 万个 （每个漏洞有一个 CVE 编号，记录成因、影响和修补办法），人工审计根本跟不上。
- **老工具不太好使**：传统静态分析工具靠写死的规则去匹配，规则没覆盖就**漏报**、规则太宽就**误报**，还说不出「为什么觉得有问题」。
- **大模型有机会，但缺两样东西**：它读得懂代码，但不知道具体漏洞长什么样（缺「安全常识」），也不知道谁调用了这个函数（缺「上下文」）。

本项目的创新点

- **不急着下结论**：像查案一样，先找证据、再说结论
- **两路取证**：既翻「漏洞案例手册」，也去仓库查调用关系
- **全程留痕**：查了什么、读了什么、为什么这么判，都在界面上

二、项目内容：我们要做一个什么东西

输入一段代码，它告诉你「有没有漏洞、为什么、怎么查出来的」

一句话说清楚

给智能体一段 C/C++ **函数代码**和它所在**代码仓库的地址**，它回答三件事：有没有漏洞 —— 凭什么这么说 —— 它是怎么查出来的。

难点一：大模型不懂安全知识

人话版：它像个聪明但不熟悉业务的实习生，得先给它一本「漏洞案例手册」。

做法：把历史上已披露的真实漏洞案例整理成可检索的知识库，每次检测先去翻这本手册，找相似的旧案例作参考。

难点二：光看一个函数判断不了

人话版：只看一页纸，判断不了整件事——得知道它被谁调用、又调用了谁。

做法：给智能体装上「去仓库里查资料」的本事，让它自己决定什么时候需要查、查什么。

我们主动把范围收窄：只做「判断 + 解释」，不做自动修复、不扫全仓库。先把流程跑通，再谈性能。

三、需求建模（1）业务需求与用户需求

谁会用、在什么场合用、用来解决什么问题

业务需求

在代码上线之前，快速把「**看着可能有漏洞**的函数」筛出来，让安全人员集中精力看真正可疑的地方，而不是把代码从头看到尾。

用户需求：谁会用它

- **安全工程师**：日常代码审计，成批排查
- **开发人员**：提交代码前自己先查一遍
- **学生 / 研究者**：了解漏洞模式，做对比实验

典型使用场景

1. 把一段函数和仓库地址贴进界面
2. 界面实时显示它每一步在查什么
3. 拿到结论：有无漏洞 + 理由 + 分析记录
4. 需要时上传文件，一次跑几十上百条

三、需求建模（2）功能需求

系统必须能做的事，一共六项

功能	说明
F1 单样本检测	贴一段函数 + 仓库地址，给出「有漏洞 / 无漏洞」的结论
F2 批量检测	上传 `.jsonl` 文件一次跑多条，汇总生成结果文件
F3 知识检索	自动去「漏洞案例手册」里找相似案例，并展示找到了哪几条
F4 上下文挖掘	自动去代码仓库查调用方、被调用方、全局变量、类型与宏
F5 报告生成	输出「判定结果 + 判定理由 + 分析记录」三件套
F6 过程可视化	界面实时展示它每一步在做什么，事后还能回看整个分析过程

六项功能围绕一条主线：看得见的检测过程——不仅要给结论，还要让人能复核这个结论。

三、需求建模（3）非功能需求

做得好不好，看这五条

- **判断要准**：核心考核标准是「同一个函数的漏洞版和修复版能否被区分开」，区分得越多分越高——而不是简单地「见谁都判有漏洞」。
- **过程要透明**：每一次知识检索、每一次仓库读取都要留痕，结论可复核、可追问。
- **合规要严格**（硬性红线）：一次只给一个版本、不给代码改动记录、不许联网搜答案。
- **成本要可控**：限制工具调用次数与文件数、缓存仓库快照，避免跑一次耗时过久或触发限流。
- **上手要容易**：必须有图形界面，不接受只能敲命令行的版本。

一句话概括我们的取向

宁要慢一点但说得清的判断，也不要快但查不到依据的结论——可解释性是这个项目的立足点。



四、系统架构初步考虑

四层结构：界面 → 大脑 → 双手 → 资料

表现层（界面）：输入函数与仓库地址、实时看分析过程、接收报告

智能体层（大脑）：想一想 → 用个工具 → 看看结果 → 再想，证据够了才下结论

工具层（双手）：① 翻漏洞案例手册 | ② 去代码仓库查资料（grep / 找调用关系）

数据层（资料）：历史漏洞案例库 | 仓库快照缓存

分层的好处：**界面、大脑、双手、资料各管一摊**，可以分开开发和替换——比如以后换一个更聪明的大模型，其余三层不用动。



五、开发技术栈与工具平台

原则：选成熟、资料多、踩坑少的技术，把力气花在真正的难点上

模块	选型	为什么选它
开发语言	Python 3.11	生态最全，前后端与算法一把梭
智能体框架	LangChain（Agent + 工具调用）	成熟、示例最多，「大脑思考」的架子现成
知识库 / 检索	Chroma 向量库 + 中文嵌入模型	本地就能跑、零服务依赖，便于复现
代码理解	GitHub API + Tree-sitter + grep/find	精确拉取指定版本代码，并画出调用关系
图形界面	Streamlit	天然支持实时刷新与文件上传，开发量小
底层大模型	兼容 OpenAI 协议的大模型	接口统一，随时可换模型做对比
协作与版本	Git + GitHub（Issue / 分支 / PR）	过程留痕，团队协作数据可查
开发工具	VS Code / PyCharm	免费、插件多，团队通用

六、Git 协作开发：过程数据可查

五个人的代码怎么合到一起，且每一步都留得下痕迹

我们的工作流

- **仓库**：GitHub 建团队仓库，代码、文档、脚本统一托管
- **分支**：`main`（随时可演示）← `dev`（集成）← `feat/xxx`（各人开发）
- **任务**：每项任务先开 Issue，认领后动手，看板上看得到进度
- **合并**：一律走 Pull Request，至少一人 review 后才合并
- **提交**：commit 写清「做了什么」，不写「update」

为什么强调「可查」

- 提交记录、Issue、PR、评审意见全部公开
- 谁做了什么、做到哪一步，一目了然
- 既是协作依据，也是个人贡献的凭证
- 出问题能回溯到具体哪次改动

七、项目开发计划与进度安排

七周里程碑，每周都有能拿得出手的东西

周次	阶段	本周产出（可验收）
W1	需求与数据摸底	理清数据字段；手工跑通几个样例，定下评分口径
W2	搭建知识库	历史案例入库，检索功能可用并能自测效果
W3	仓库查询工具	能拉取指定版本代码，能解析函数调用关系
W4	打通智能体	「先检索 → 再判断 → 需要时查仓库」的完整流程跑通
W5	界面与批量	图形界面可用（输入 / 过程 / 报告）+ 批量文件检测
W6	联调调优与评测	300 条一起跑，按评价标准统计并迭代优化
W7	报告与验收	技术报告、系统演示、结果文件提交

节奏：每周一次组会同步进度；从 W4 开始，保证「任何时候都能当场演示一个完整流程」。

八、项目团队组成与工作安排

五人各管一摊，接口先定好再并行开发

- **李柏谚（组长）**：总体架构与智能体主流程；把控进度、协调接口
- **杨聃**：历史漏洞案例的知识库与检索（把「手册」做出来并保证查得准）
- **王义卓**：仓库查询工具（拉取代码、解析调用关系、封装检索命令）
- **孙伊凡**：图形界面与批量流程（过程可视化、结果展示）
- **徐培栋**：评测、集成与文档（跑批统计、回归测试、报告撰写）

协作方式

数据接口先行——先把「一条案例长什么样、工具返回什么、报告有哪些字段」定死，各人再并行开发，最后统一集成；全员参与联调与评测。

九、预期成果与评价目标

最后交付什么，以及我们打算拿到什么水平

交付物

- 一个能跑的完整系统：图形界面 + 知识库 + 仓库查询工具
- 300 条样本的检测结果文件
- 技术报告与现场演示

我们的目标

核心看「**同一函数的漏洞版与修复版能否被正确区分**」：**争取 B 级及以上**（区分对 **25 组以上**），同时把「**完全判反**」的情况压到区分对数的一半以内。

可能踩的坑与预案

- **把修复版也判成有漏洞**：提高门槛，必须能指出具体可疑点才判「有漏洞」
- **把漏洞版判成没问题**：加强案例手册的提示作用，放宽初筛
- **接口被限流**：仓库代码本地缓存，不重复拉取
- **进度紧张**：先保主流程能演示，界面细节往后放